Binary Search

arr = [- - - -] (Linear Search)

Police →
- Photo → Target
- Last seen location → Search space

Binary Search

- Target
- Search space
- Condition to discard one half of search space.

arr = [- - - . . . - - - .] ←.

Reduced Search Space ↑ Not present

Mechanism

- Go to the middle element
- Apply the condition and discard one half.
- Repeat the steps with the new search space.

Question: Given a sorted array with distinct elements, search for the index of k. If k is not present, return -1

0	1	2	3	4	5	6	7	8	9

ex: arr = [3, 6, 9, 12, 14, 19, 20, 23, 25, 27]

k = 10 ⇒ O/P = -1

k = 9 ⇒ O/P = 2

Brute Force Approach

- Linear search.

TC = O(N) SC = O(1)

Binary Search

1) Target → k.
2) Search space → Array.
3) Condition → Sorted Array, discard on magnitude

0	1	2	3	4	5	6	7	8	9

arr = [3, 6, 9, 12, 14, 19, 20, 23, 25, 27]

Discard.

k = 15

1) If arr[mid] > k ⇒ Go left.
2) If arr[mid] < k ⇒ Go right.
3) If arr[mid] == k ⇒ Return mid.

	0	1	2	3	4	5	6	7	8	9
arr =	3,	6,	9,	12,	14,	19,	20,	23,	25,	27

d m h

k = 12

```
def binarySearch(arr, k):
    low = 0, high = len(arr) - 1
    while (low <= high):
        mid = (low + high) // 2
        if(arr[mid] > k):   # Go left
            high = mid - 1
        elif(arr[mid] < k):   # Go right.
            low = mid + 1
        else: return mid.
    # Not found
    return -1
```

$N \to N/2 \to N/4 \to \cdots 1$

TC = $O(\log N)$ SC = $O(1)$

Question: Given a sorted array of N elements, find the first occurrence of the target element.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
ex: arr =	-5	-5	-3	0	0	1	1	$\boxed{5}$	5	5	8	10	10	15	15
	l							m							h

$k = 5 \Rightarrow O/P = 7$

$k = 0 \Rightarrow O/P = 3$

$k = -5 \Rightarrow O/P = 0$

Approach 1 - Check left element if arr[mid] == k

```
def binarySearch(arr, k):
    low = 0, high = len(arr) - 1
    while (low <= high):
        mid = (low + high) // 2
        if(arr[mid] > k): high = mid - 1
        elif(arr[mid] < k): low = mid + 1
        else:
            if (mid>0 and arr[mid-1] == arr[mid]):
                high=mid-1
            else: return mid.
    return -1
```

Approach 2 - Even after getting the answer, store it and go left.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
ex: arr =	-5	-5	-3	0	0	1	1	5	5	$\boxed{5}$	8	10	10	15	15
	l								h	m					h

ans = ~~4~~ ~~8~~ 7

```
def binarySearch(arr, k):
    low = 0, high = len(arr) - 1
    ans = -1
    while (low <= high):
        mid = (low + high) // 2
        if(arr[mid] > k):    # Go left
            high = mid - 1
        elif(arr[mid] < k):  # Go right.
            low = mid + 1
        else:
            ans = mid
            high = mid - 1
    # Not found
    return ans
```

TC = O(logN)

SC = O(1)

Question: Every element occurs twice except for one, find the unique element. Also, duplicate elements are adjacent to each other but array is not sorted.

ex: arr = [8 8 5 5 6 2 2]

0 1 2 3 4 5 6

l h m h

Brute Force Approach

- Frequency map.
- Check for element with freq == 1.

TC = O(N) SC = O(N).

arr = [- - - - 6 6 - - -]

5

m

Condition to discard

- If unique (!= prev , != next) : return
- If FO (first occurrence) is odd index : Move left
- If FO is even index : Move right.

```
def uniqueElement(arr):
    l=0, h=len(arr)-1
    while(l<=h):
        m=(l+h)//2
        # if unique.
        if(m>0 and arr[m]!=arr[m-1] and
            m<len(arr)-1 and arr[m]!=arr[m+1]):
            return arr[m]
        # Get the fO (First occurrence)
        fo=m
        if(arr[m]==arr[m-1]): fo=m-1
        # Go left.
        if(fo%2!=0): h=m-1
        else: l=m+1
    return -1
```

TC = O(logN) SC = O(1)